

Q1. Smart Document Editor using String, StringBuilder and StringBuffer

CODE

```
import java.util.Scanner;

public class Q1_SmartDocumentEditor {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.println("Enter paragraph (END to finish):");
        String paragraph = readParagraph(sc);

        int charCount = countCharacters(paragraph);
        int wordCount = countWords(paragraph);
        int sentenceCount = countSentences(paragraph);

        System.out.print("Target word: ");
        String target = sc.next();
        System.out.print("Replacement word: ");
        String replacement = sc.next();

        String replacedDoc = paragraph.replace(target, replacement);
        String reversedSentences = reverseEachSentence(replacedDoc);

        StringBuffer history = new StringBuffer();
        history.append("Replacements done: ").append(target)
            .append("' -> '").append(replacement).append("\n");
        history.append("Sentences reversed.\n");
        history.append("Counts -> Chars: ").append(charCount)
            .append(", Words: ").append(wordCount)
            .append(", Sentences: ").append(sentenceCount).append("\n");

        StringBuilder finalDoc = new StringBuilder();
        finalDoc.append("----- ORIGINAL -----\n");
        finalDoc.append(paragraph).append("\n\n");

        finalDoc.append("----- EDIT HISTORY -----\n");
        finalDoc.append(history).append("\n");

        finalDoc.append("----- FINAL -----\n");
        finalDoc.append("Characters: ").append(charCount).append("\n");
        finalDoc.append("Words: ").append(wordCount).append("\n");
        finalDoc.append("Sentences: ").append(sentenceCount).append("\n");
    }

    private static String readParagraph(Scanner sc) {
        StringBuilder sb = new StringBuilder();
        while (true) {
            String line = sc.nextLine();
            if (line.equals("END")) break;
            sb.append(line).append("\n");
        }
        return sb.toString();
    }

    private static int countCharacters(String str) {
        return str.length();
    }

    private static int countWords(String str) {
        return str.trim().split("\\s+").length;
    }

    private static int countSentences(String str) {
        return str.trim().split("\\.\\?\\!").length;
    }

    private static String reverseEachSentence(String str) {
        return str.trim().split("\\.\\?\\!").length > 1 ? str : str;
    }
}
```

```

System.out.println("Enter paragraph (END to finish):");
String paragraph = readParagraph(sc);

int charCount = countCharacters(paragraph);
int wordCount = countWords(paragraph);
int sentenceCount = countSentences(paragraph);

System.out.print("Target word: ");
String target = sc.next();
System.out.print("Replacement word: ");
String replacement = sc.next();

String replacedDoc = paragraph.replace(target, replacement);
String reversedSentences = reverseEachSentence(replacedDoc);

StringBuffer history = new StringBuffer();
history.append("Replacements done: ").append(target)
    .append("' ' -> ' ').append(replacement).append("\n");
history.append("Sentences reversed.\n");
history.append("Counts -> Chars: ").append(charCount)
    .append(", Words: ").append(wordCount)
    .append(", Sentences: ").append(sentenceCount).append("\n");

StringBuilder finalDoc = new StringBuilder();
finalDoc.append("----- ORIGINAL -----\n");
finalDoc.append(paragraph).append("\n\n");

finalDoc.append("----- EDIT HISTORY -----\n");
finalDoc.append(history).append("\n");

finalDoc.append("----- FINAL -----\n");
finalDoc.append("Characters: ").append(charCount).append("\n");
finalDoc.append("Words: ").append(wordCount).append("\n");
finalDoc.append("Sentences: ").append(sentenceCount).append("\n\n");
finalDoc.append(reversedSentences);

System.out.println(finalDoc.toString());
sc.close();
}

private static String readParagraph(Scanner sc) {
    StringBuilder sb = new StringBuilder();

    while (true) {
        String line = sc.nextLine();
        if (line.equals("END")) break;
        sb.append(line).append(" ");
    }
    return sb.toString().trim();
}

private static int countCharacters(String s) {
    int count = 0;
    for (int i = 0; i < s.length(); i++) {
        if (s.charAt(i) != ' ') count++;
    }
}

```

SIMATS Engineering • Department of CSE • CSA0925 — Programming In java • 14-08-2026 • Category: CO2-AT1-Assignment1 • Student Submissions

Report

```

    }
    return count;
}

private static int countWords(String s) {
    s = s.trim();
    if (s.isEmpty()) return 0;
    return s.split("\\s+").length;
}

private static int countSentences(String s) {
    int count = 0;
    for (int i = 0; i < s.length(); i++) {
        char c = s.charAt(i);
        if (c == '.' || c == '!' || c == '?') count++;
    }
    return count;
}

private static String reverseEachSentence(String doc) {
    StringBuilder out = new StringBuilder();
    StringBuilder current = new StringBuilder();

    for (int i = 0; i < doc.length(); i++) {
        char ch = doc.charAt(i);
        current.append(ch);

        if (ch == '.' || ch == '!' || ch == '?') {
            String sentence = current.toString();
            char delimiter = sentence.charAt(sentence.length() - 1);
            String content = sentence.substring(0, sentence.length() - 1);

            String reversedContent = new StringBuilder(content).reverse().toString();
            out.append(reversedContent).append(delimiter).append(" ");

            current.setLength(0);
        }
    }

    if (current.length() > 0) {
        out.append(new StringBuilder(current.toString()).reverse().toString());
    }

    return out.toString().trim();
}
}

```

INPUT

```

Hello world. Welcome to Java programming! Hello everyone?
This is a small test program.
END
Hello
HI

```

OUTPUT

(no output)

```
import java.util.ArrayList;
import java.util.List;
import java.util.Scanner;

public class HospitalTokenSystem {
    private static List<Patient> list = new ArrayList<>();

    public static void main(String[] args) {
        reg("P101", "Alice", "Cardio", 45);
        reg("P102", "Bob", "Neuro", 34);
        reg("P103", "Charlie", "Cardio", 60);

        showAll();
        update("P102", "Robert", "Neuro", 35);
        showDept("Cardio");
        cancel("P103");
        showAll();
        showFirstLast();
    }

    public static void reg(String id, String name, String dept, int age) {
        list.add(new Patient(id, name, dept, age));
        System.out.println("Registered. Token: " + list.size());
    }

    public static void showAll() {
        if (list.isEmpty()) return;
        for (int i = 0; i < list.size(); i++) {
            System.out.println("Token " + (i + 1) + ": " + list.get(i));
        }
    }

    public static void update(String id, String name, String dept, int age) {
        for (Patient p : list) {
            if (p.id.equalsIgnoreCase(id)) {
                p.name = name;
                p.dept = dept;
                p.age = age;
                return;
            }
        }
    }

    public static void cancel(String id) {
        list.removeIf(p -> p.id.equalsIgnoreCase(id));
    }

    public static void showDept(String dept) {
        for (Patient p : list) {
            if (p.dept.equalsIgnoreCase(dept)) System.out.println(p);
        }
    }

    public static void showFirstLast() {
        if (list.isEmpty()) {
            System.out.println("First: " + list.get(0));
            System.out.println("Last: " + list.get(list.size() - 1));
        }
    }
}
```

Report

```
class Patient {
    String id, name, dept;
    int age;

    Patient(String id, String name, String dept, int age) {
        this.id = id;
        this.name = name;
        this.dept = dept;
        this.age = age;
    }

    public String toString() {
        return id + " | " + name + " | " + dept + " | " + age;
    }
}
```

INPUT

P101 Alice Cardio 45
P102 Bob Neuro 34
P103 Charlie Cardio 60
P102 Robert Neuro 35
Cardio
P103

OUTPUT

(no output)

Q3. Airport Boarding Queue Management using Queue Interface

CODE

```
import java.util.*;

public class AirportBoardingQueue {

    static Queue<String> queue = new LinkedList<>();
    static PriorityQueue<String> vipQueue = new PriorityQueue<>();

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);
        boolean run = true;

        while (run) {
            int ch = sc.nextInt();

            switch (ch) {

                case 1:
                    addPassenger(sc);
                    break;

                case 2:
                    displayQueue();
                    break;

                case 3:
                    boardPassenger();
                    break;

                case 4:
                    nextPassenger();
                    break;

                case 5:
                    addVIP(sc);
                    break;

                case 6:
                    System.out.println("Thank You");
                    run = false;
                    break;

                default:
                    System.out.println("Invalid Choice");
            }
        }

        sc.close();
    }

    static void addPassenger(Scanner sc) {
        System.out.print("Enter Passenger Name: ");
        String name = sc.next();
        queue.offer(name);
        System.out.println("Passenger Added");
    }

    static void displayQueue() {
        System.out.println("VIP Queue: " + vipQueue);
        System.out.println("Normal Queue: " + queue);
    }
}
```

SIMATS Engineering • Department of CSE • CSA0925 — Programming In java • 14-08-2026 • Category: CO2-AT1-Assignment1 • Student Submissions Report

```
static void boardPassenger() {

    if (!vipQueue.isEmpty()) {
        System.out.println("Boarded VIP Passenger: " + vipQueue.poll());
    } else if (!queue.isEmpty()) {
        System.out.println("Boarded Passenger: " + queue.poll());
    } else {
        System.out.println("Queue Empty");
        return;
    }

    System.out.println("Remaining VIP Queue: " + vipQueue);
    System.out.println("Remaining Normal Queue: " + queue);
}

static void nextPassenger() {

    if (!vipQueue.isEmpty()) {
        System.out.println("Next VIP Passenger: " + vipQueue.peek());
    } else if (!queue.isEmpty()) {
        System.out.println("Next Passenger: " + queue.peek());
    } else {
        System.out.println("Queue Empty");
    }
}

static void addVIP(Scanner sc) {
    System.out.print("Enter VIP Passenger Name: ");
    String name = sc.next();
    vipQueue.offer(name);
    System.out.println("VIP Passenger Added");
}
}
```

INPUT

```
1
Rahul
1
Priya
5
Arjun
2
4
3
3
2
6
```

OUTPUT

(no output)

Q4. Digital Certificate Verification using Set Interface

CODE

```
import java.util.*;

public class DigitalCertificateVerification {

    static LinkedHashSet<String> certificates = new LinkedHashSet<>();

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);
        boolean run = true;

        while (run) {
            int ch = sc.nextInt();

            switch (ch) {

                case 1:
                    addCertificate(sc);
                    break;

                case 2:
                    verifyCertificate(sc);
                    break;

                case 3:
                    displayInsertionOrder();
                    break;

                case 4:
                    displaySortedOrder();
                    break;

                case 5:
                    totalCertificates();
                    break;

                case 6:
                    removeCertificate(sc);
                    break;

                case 7:
                    System.out.println("Thank You");
                    run = false;
                    break;

                default:
                    System.out.println("Invalid Choice");
            }
        }

        sc.close();
    }

    static void addCertificate(Scanner sc) {
        String id = sc.next();

        if (certificates.add(id))
            System.out.println("Certificate Added");
        else
            System.out.println("Duplicate Certificate Not Allowed");
    }
}
```

SIMATS Engineering • Department of CSE • CSA0925 — Programming In java • 14-08-2026 • Category: CO2-AT1-Assignment1 • Student Submissions
Report

```
static void verifyCertificate(Scanner sc) {
    String id = sc.next();

    if (certificates.contains(id))
        System.out.println("Certificate Exists");
    else
        System.out.println("Certificate Not Found");
}

static void displayInsertionOrder() {
    System.out.println("Certificates:");
    for (String id : certificates)
        System.out.println(id);
}

static void displaySortedOrder() {
    TreeSet<String> sorted = new TreeSet<>(certificates);

    System.out.println("Sorted Certificates:");
    for (String id : sorted)
        System.out.println(id);
}

static void totalCertificates() {
    System.out.println("Total Unique Certificates: " + certificates.size());
}

static void removeCertificate(Scanner sc) {
    System.out.print("Enter Certificate ID: ");
    String id = sc.next();

    if (certificates.remove(id))
        System.out.println("Certificate Removed");
    else
        System.out.println("Certificate Not Found");
}
}
```

INPUT

```
1
C101
1
C102
1
C103
1
C101
2
C102
```

Q5. Smart Inventory Management using Map Interface

CODE

```
import java.util.*;

public class SmartInventoryManagement {

    static HashMap<Integer, Product> inventory = new HashMap<>();

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);
        boolean run = true;

        while (run) {
            int ch = sc.nextInt();

            switch (ch) {

                case 1:
                    addProduct(sc);
                    break;

                case 2:
                    updateQuantity(sc);
                    break;

                case 3:
                    searchProduct(sc);
                    break;

                case 4:
                    totalValue();
                    break;

                case 5:
                    displayCategoryWise();
                    break;

                case 6:
                    removeProduct(sc);
                    break;

                case 7:
                    displayProducts();
                    break;

                case 8:
                    System.out.println("Thank You");
                    run = false;
                    break;

                default:
                    System.out.println("Invalid Choice");
            }
        }

        sc.close();
    }

    static void addProduct(Scanner sc) {

        System.out.print("Enter Product ID: ");
        int id = sc.nextInt();
```

```
        System.out.print("Enter Product Name: ");
        String name = sc.next();

        System.out.print("Enter Category: ");
        String category = sc.next();

        System.out.print("Enter Quantity: ");
        int qty = sc.nextInt();

        System.out.print("Enter Price: ");
        double price = sc.nextDouble();

        inventory.put(id, new Product(id, name, category, qty, price));

        System.out.println("Product Added");
    }

    static void updateQuantity(Scanner sc) {

        System.out.print("Enter Product ID: ");
        int id = sc.nextInt();

        if (inventory.containsKey(id)) {
            System.out.print("Enter New Quantity: ");
            int qty = sc.nextInt();
            inventory.get(id).quantity = qty;
            System.out.println("Quantity Updated");
        } else {
            System.out.println("Product Not Found");
        }
    }

    static void searchProduct(Scanner sc) {

        System.out.print("Enter Product ID: ");
        int id = sc.nextInt();

        if (inventory.containsKey(id))
            System.out.println(inventory.get(id));
        else
            System.out.println("Product Not Found");
```

```

        System.out.print("Enter Product Name: ");
        String name = sc.next();

        System.out.print("Enter Category: ");
        String category = sc.next();

        System.out.print("Enter Quantity: ");
        int qty = sc.nextInt();

        System.out.print("Enter Price: ");
        double price = sc.nextDouble();

        inventory.put(id, new Product(id, name, category, qty, price));

        System.out.println("Product Added");
    }

    static void updateQuantity(Scanner sc) {

        System.out.print("Enter Product ID: ");
        int id = sc.nextInt();

        if (inventory.containsKey(id)) {
            System.out.print("Enter New Quantity: ");
            int qty = sc.nextInt();
            inventory.get(id).quantity = qty;
            System.out.println("Quantity Updated");
        } else {
            System.out.println("Product Not Found");
        }
    }

    static void searchProduct(Scanner sc) {

        System.out.print("Enter Product ID: ");
        int id = sc.nextInt();

        if (inventory.containsKey(id))
            System.out.println(inventory.get(id));
        else
            System.out.println("Product Not Found");
    }

    static void totalValue() {

        double total = 0;

        for (Product p : inventory.values())
            total += p.quantity * p.price;

        System.out.println("Total Inventory Value: " + total);
    }

    static void displayCategoryWise() {

        for (Product p : inventory.values())
            System.out.println("Category: " + p.category + " -> " + p);
    }

    static void removeProduct(Scanner sc) {

        System.out.print("Enter Product ID: ");
        int id = sc.nextInt();

```

```

        if (inventory.remove(id) != null)
            System.out.println("Product Removed");
        else
            System.out.println("Product Not Found");
    }

    static void displayProducts() {

        if (inventory.isEmpty()) {
            System.out.println("No Products");
            return;
        }

        for (Product p : inventory.values())
            System.out.println(p);
    }
}

class Product {

    int id;
    String name;
    String category;
    int quantity;
    double price;

    Product(int id, String name, String category, int quantity, double price) {
        this.id = id;
        this.name = name;
        this.category = category;
        this.quantity = quantity;
        this.price = price;
    }

    public String toString() {
        return "ID: " + id +

```

```
System.out.print("Enter Product ID: ");
int id = sc.nextInt();
```

SIMATS Engineering • Department of CSE • CSA0925 — Programming In java • 14-08-2026 • Category: CO2-AT1-Assignment1 • Student Submissions
Report

```
        if (inventory.remove(id) != null)
            System.out.println("Product Removed");
        else
            System.out.println("Product Not Found");
    }

    static void displayProducts() {

        if (inventory.isEmpty()) {
            System.out.println("No Products");
            return;
        }

        for (Product p : inventory.values())
            System.out.println(p);
    }
}

class Product {

    int id;
    String name;
    String category;
    int quantity;
    double price;

    Product(int id, String name, String category, int quantity, double price) {
        this.id = id;
        this.name = name;
        this.category = category;
        this.quantity = quantity;
        this.price = price;
    }

    public String toString() {
        return "ID: " + id +
            " Name: " + name +
            " Category: " + category +
            " Quantity: " + quantity +
            " Price: " + price;
    }
}
```

INPUT

```
1
101
Laptop
Electronics
5
50000
1
102
Mouse
Electronics
10
500
7
3
101
2
101
8
4
5
6
102
7
8
```

SIMATS Engineering • Department of CSE • CSA0925 — Programming In java • 14-08-2026 • Category: CO2-AT1-Assignment1 • Student Submissions
Report

OUTPUT (no output)